```
        {
                if(i%2 == 0)
                        msg_priority = 0;
                else
                        msg_priority = 10;

                sprintf(&tx_buffer[14], "%d", i);

                printf("Sender: sending message: [%s] with
priority: [%d]\n", tx_buffer, msg_priority);
                // send message to message queue
                if(mq_send(mqd_sender, &tx_buffer[0], strlen
(tx_buffer), msg_priority) == (mqd_t)-1)
                {
                        perror("Sender: Unable to send
message");
                }
        }

        if(mq_close(mqd_sender) == (mqd_t)-1)
        {
                perror("Sender: mq_close");
                exit(1);
        }
        printf("Sender: Exit\n");

        exit(0);
}
```

// pmq_receiver_priority.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mqueue.h>

#define MQ_PERMISSIONS 0660
#define MQ_MAX_MESSAGES 10
#define MQ_MAX_MSG_SIZE 128
#define MQ_MSG_BUFFER_SIZE MQ_MAX_MSG_SIZE + 10

int main(int argc, char **argv)
{
        mqd_t mqd_receiver;
        const char *mq_name = "/mq-postman";
        struct mq_attr attr;
        unsigned int msg_priority;
        char rx_buffer[MQ_MSG_BUFFER_SIZE];

        attr.mq_flags = 0;
        attr.mq_maxmsg = MQ_MAX_MESSAGES;
        attr.mq_msgsize = MQ_MAX_MSG_SIZE;
        attr.mq_curmsgs = 0;

        if((mqd_receiver = mq_open(mq_name, O_RDONLY | O_
CREAT, MQ_PERMISSIONS, &attr)) == (mqd_t)-1)
        {
                perror("Receiver: mq_open");
                exit(1);
        }
        else
        {
                printf("Receiver MQ %s opened\n", mq_name);
        }

        int i=9;
        while(i)
        {
                printf("Receiver: blocked, waiting for
messages...\n");
                // get the highest priority message first
                if(mq_receive(mqd_receiver, rx_buffer, MQ_
MSG_BUFFER_SIZE, &msg_priority) == (mqd_t)-1)
                {
                        perror("Sender: mq_receive");
                        exit(1);
                }
                else
                {
                        printf("Receiver: message received:
[%s] with priority: [%d]\n", rx_buffer, msg_priority);
                        i--;
                }
        }

        if(mq_close(mqd_receiver) == (mqd_t)-1)
        {
                perror("Receiver: mq_close");
                exit(1);
        }

        printf("Receiver: Exit\n");

        exit(0);
}
```

In the above example, run the *pmq_receiver_priority* first in a terminal, followed by *pmq_sender_priority* in another terminal.

As demonstrated in Figures 6 and 7, the sender process sends 9 messages to the message queue, with message